

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"



**АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ КОМП'ЮТЕРНИХ
СИСТЕМ**

Лабораторна робота № 5

Виконав: ст. гр. КІ-404

Грицай В.О.

Прийняв:
Федак П.Р.

Львів 2024

Тема: ознайомлення з автоматичним тестуванням.

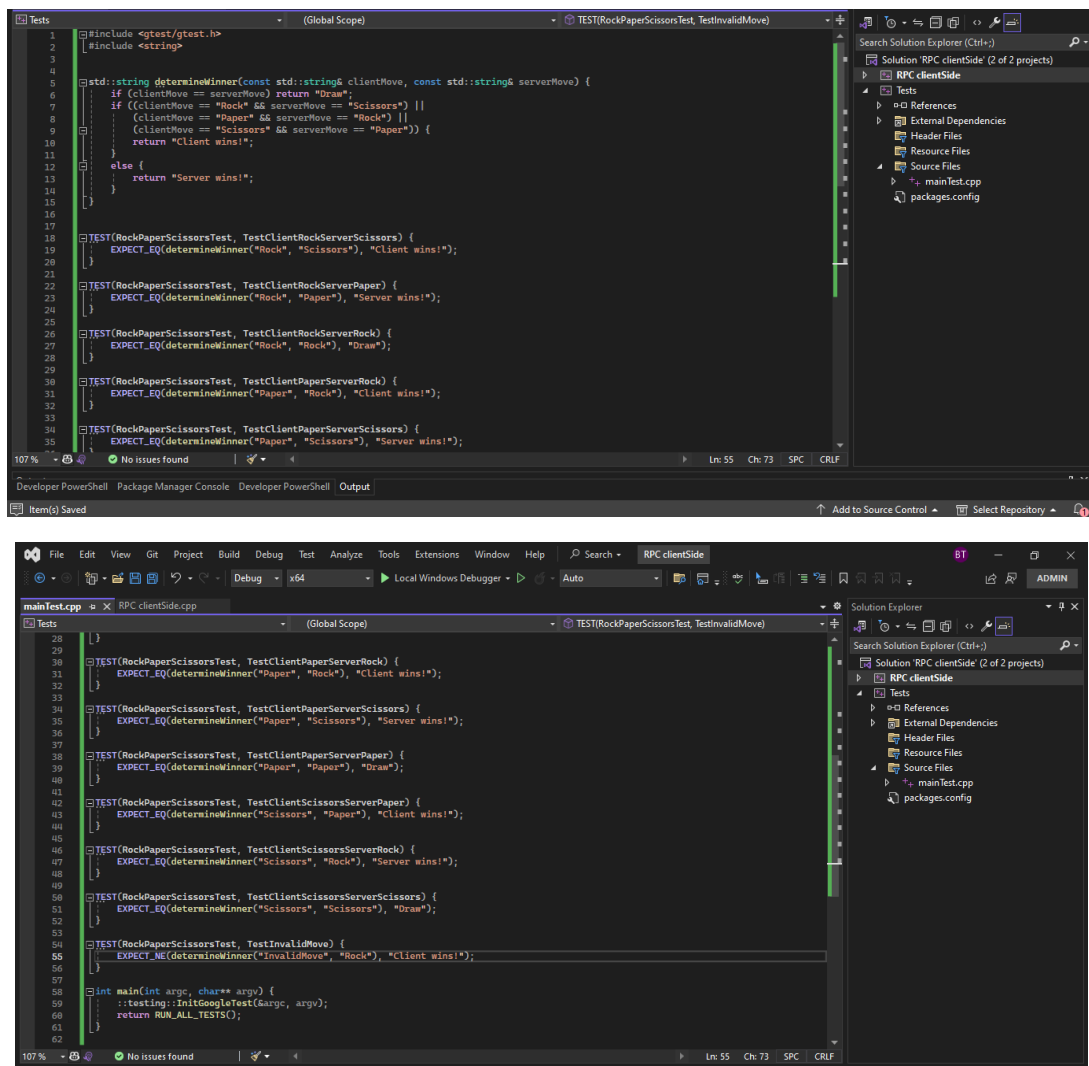
Завдання для виконання лабораторної роботи:

1. Implement or use existing test framework;
2. Create a set of automated tests;
3. Test report should contain number of all tests, passed tests, failed tests, coverage;
4. Coverage must be more than 80;

Хід виконання роботи:

Для тестування програми написаної на мові C++, я вибрав фреймворк Google test.

Код тестового файлу mainTest.cpp :



The image consists of two screenshots of the Visual Studio IDE. The top screenshot shows a file named 'mainTest.cpp' with the following code:

```
1 #include <gtest/gtest.h>
2 #include <string>
3
4
5 std::string determineWinner(const std::string& clientMove, const std::string& serverMove) {
6     if (clientMove == serverMove) return "Draw";
7     if (clientMove == "Rock" && serverMove == "Scissors") ||
8     if (clientMove == "Paper" && serverMove == "Rock") ||
9     if (clientMove == "Scissors" && serverMove == "Paper") {
10         return "Client wins!";
11     }
12     else {
13         return "Server wins!";
14     }
15 }
16
17 TEST(RockPaperScissorsTest, TestClientRockServerScissors) {
18     EXPECT_EQ(determineWinner("Rock", "Scissors"), "Client wins!");
19 }
20
21 TEST(RockPaperScissorsTest, TestClientRockServerPaper) {
22     EXPECT_EQ(determineWinner("Rock", "Paper"), "Server wins!");
23 }
24
25 TEST(RockPaperScissorsTest, TestClientRockServerRock) {
26     EXPECT_EQ(determineWinner("Rock", "Rock"), "Draw");
27 }
28
29 TEST(RockPaperScissorsTest, TestClientPaperServerRock) {
30     EXPECT_EQ(determineWinner("Paper", "Rock"), "Client wins!");
31 }
32
33 TEST(RockPaperScissorsTest, TestClientPaperServerScissors) {
34     EXPECT_EQ(determineWinner("Paper", "Scissors"), "Server wins!");
35 }
36
37 TEST(RockPaperScissorsTest, TestClientPaperServerPaper) {
38     EXPECT_EQ(determineWinner("Paper", "Paper"), "Draw");
39 }
40
41 TEST(RockPaperScissorsTest, TestClientScissorsServerPaper) {
42     EXPECT_EQ(determineWinner("Scissors", "Paper"), "Client wins!");
43 }
44
45 TEST(RockPaperScissorsTest, TestClientScissorsServerRock) {
46     EXPECT_EQ(determineWinner("Scissors", "Rock"), "Server wins!");
47 }
48
49 TEST(RockPaperScissorsTest, TestClientScissorsServerScissors) {
50     EXPECT_EQ(determineWinner("Scissors", "Scissors"), "Draw");
51 }
52
53 TEST(RockPaperScissorsTest, TestInvalidMove) {
54     EXPECT_NE(determineWinner("InvalidMove", "Rock"), "Client wins!");
55 }
56
57 int main(int argc, char* argv[]) {
58     ::testing::InitGoogleTest(&argc, argv);
59     return RUN_ALL_TESTS();
60 }
```

The bottom screenshot shows the same file 'mainTest.cpp' with the following code:

```
28 }
29
30 TEST(RockPaperScissorsTest, TestClientPaperServerRock) {
31     EXPECT_EQ(determineWinner("Paper", "Rock"), "Client wins!");
32 }
33
34 TEST(RockPaperScissorsTest, TestClientPaperServerScissors) {
35     EXPECT_EQ(determineWinner("Paper", "Scissors"), "Server wins!");
36 }
37
38 TEST(RockPaperScissorsTest, TestClientPaperServerPaper) {
39     EXPECT_EQ(determineWinner("Paper", "Paper"), "Draw");
40 }
41
42 TEST(RockPaperScissorsTest, TestClientScissorsServerPaper) {
43     EXPECT_EQ(determineWinner("Scissors", "Paper"), "Client wins!");
44 }
45
46 TEST(RockPaperScissorsTest, TestClientScissorsServerRock) {
47     EXPECT_EQ(determineWinner("Scissors", "Rock"), "Server wins!");
48 }
49
50 TEST(RockPaperScissorsTest, TestClientScissorsServerScissors) {
51     EXPECT_EQ(determineWinner("Scissors", "Scissors"), "Draw");
52 }
53
54 TEST(RockPaperScissorsTest, TestInvalidMove) {
55     EXPECT_NE(determineWinner("InvalidMove", "Rock"), "Client wins!");
56 }
57
58 int main(int argc, char* argv[]) {
59     ::testing::InitGoogleTest(&argc, argv);
60     return RUN_ALL_TESTS();
61 }
```

Результати виконання тестів:

```
Microsoft Visual Studio Debug Console
Test 1 Passed! Time execution 0.1 s
Test 2 Passed! Time execution 0.2 s
Test 3 Passed! Time execution 0.3 s
Test 4 Passed! Time execution 0.4 s
Test 5 Passed! Time execution 0.5 s
Test 6 Passed! Time execution 0.6 s
Test 7 Passed! Time execution 0.7 s
Test 8 Passed! Time execution 0.8 s
Test 9 Passed! Time execution 0.9 s
Test 10 Passed! Time execution 1.0 s

C:\Users\HP\source\repos\RPC clientSide\x64\Debug\RPC clientSide.exe (process 17096) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the co
le when debugging stops.
Press any key to close this window . . .
```

Як бачимо всі тести пройшли успішно!

Далі використав утиліту Coverlet для формування звіту про тестування :

```
PS C:\Users\HP\source\repos\RPS SW-HW game> reportgenerator -reports:TestResults/*.json -targetdir:coveragereport -reporttypes:Html
>>
2024-10-13T01:35:34: Arguments
2024-10-13T01:35:34: -reports:TestResults/*.json
2024-10-13T01:35:34: -targetdir:coveragereport
2024-10-13T01:35:34: -reporttypes:Html
2024-10-13T01:35:35: The report file pattern 'TestResults/*.json' found no matching files.
```

Відсоток покриття тестами клієнта і сервера:

GCC Code Coverage Report

Directory: ./
Date: 2024-11-24 22:10:30
Coverage: low: ≥ 0% medium: ≥ 82.0% high: ≥ 90.0%
Exec Total Coverage
Lines: 423 541 83.0%
Functions: 456 345 82.0%

List of functions

File	Lines	Functions
client/src/client.cpp	83% 42 / 45 98.0% 5 / 5	
client/src/server.cpp	82% 41 / 43 99.0% 5 / 5	

Generated by: [GCOVR \(Version 8.2\)](#)

Висновок:

В рамках цієї лабораторної роботи було реалізовано програму для перевірки і тестування гри "Камінь, ножиці, папір". Серверна частина була реалізована на Arduino IDE, що дозволило створити простий сервер для обробки команд клієнта, таких як "LED_ON" і "LED_OFF". На клієнтській стороні було написано C++ програму для Visual Studio, яка використовувала послідовне з'єднання для надсилання команд серверу та отримання відповідей.

Окрім основного функціоналу, було створено тестовий проєкт для перевірки різних можливих сценаріїв гри. Використовуючи Google Test, було реалізовано 10 тестів, які моделювали реальне тестування гри і забезпечували перевірку правильності відповідей на кожен хід клієнта та сервера.